

Implementazione di QUIC DATAGRAM (RFC 9221) in INET

per il trasporto inaffidabile dei dati-sensore nel simulatore TOD-over-5G
(CARLA + OMNeT++/Simu5G)

Progetto TOD — Simulazione Tele-Operated Driving su 5G

20 luglio 2026

Sommario

Questo documento descrive in modo completo l'estensione del modulo di trasporto **QUIC di INET 4.6.0** con i **DATAGRAM frame** definiti dalla RFC 9221, realizzata per sostituire gli stream QUIC affidabili con un canale *inaffidabile e non ordinato* nel trasporto dei dati-sensore del veicolo tele-operato. Vengono documentate tutte le classi e tutti i metodi introdotti o modificati, il meccanismo che garantisce l'assenza di ritrasmissione, la negoziazione del parametro `max_datagram_frame_size`, le applicazioni di test, i risultati della verifica a runtime e la terminologia di rete usata.

Indice

1	Contesto e motivazione	2
2	Terminologia	2
3	Architettura: il percorso end-to-end	3
4	Modifiche a INET: classi e metodi	4
4.1	Nuovo tipo di frame (<code>packet/FrameHeader.msg</code>)	4
4.2	<code>QuicDatagramFrame</code> (<code>packet/QuicDatagramFrame.{h,cc}</code>) — nuova	4
4.3	<code>Connection</code> (<code>Connection.{h,cc}</code>)	4
4.4	<code>PacketBuilder</code> (<code>PacketBuilder.{h,cc}</code>)	5
4.5	Politica di invio: priorità e coda	5
4.6	<code>ConnectionState</code> / <code>EstablishedConnectionState</code> (<code>connectionstate/</code>)	5
4.7	<code>AppSocket</code> (<code>AppSocket.{h,cc}</code>)	6
4.8	<code>QuicSocket</code> + <code>QuicCommand.msg</code> (contract) — API applicativa	6
4.9	<code>TransportParameters</code> e <code>Quic.ned</code> — negoziazione	6
5	Il meccanismo di non-ritrasmissione	6
6	Applicazioni di test e rete	6
6.1	<code>QuicDatagramSender</code>	7
6.2	<code>QuicDatagramReceiver</code>	7
6.3	Rete <code>DatagramTest.ned</code>	7
7	Risultati della verifica	7
8	Build ed esecuzione	7
9	Versioning e prossimi passi	8

1 Contesto e motivazione

Il simulatore TOD (Tele-Operated Driving) accoppia **CARLA** (mondo fisico e percezione) e **OM-NeT++/Simu5G** (rete 5G) tramite la libreria CARLNeT. Il veicolo campiona i sensori (camera, LiDAR, radar) e li invia, attraverso la rete 5G, all'agente tele-operatore remoto, che calcola il comando di guida. La metrica chiave è il **loss_ratio**: la frazione di dati-sensore persi per ogni *sensor frame* (un campionamento a un dato istante).

Il problema. Nella versione precedente ogni sensore inviava i propri dati su uno **stream QUIC dedicato e persistente**. Poiché QUIC garantisce consegna *affidabile e ordinata* all'interno di ogni stream, la perdita di un singolo pacchetto **blocca** (head-of-line blocking) tutti i frame successivi di quel sensore finché non arriva la ritrasmissione. Di conseguenza una singola perdita si propaga a più *sensor frame* consecutivi, gonfiando artificialmente il **loss_ratio** con perdite correlate. Per uno streaming di sensori in tempo reale ciò non è realistico: un frame ritrasmesso arriva ormai scaduto e inutile, ma intanto ha bloccato quelli nuovi.

La soluzione. Introdurre i **QUIC DATAGRAM frame** (RFC 9221): un servizio di consegna *inaffidabile, non ordinato e senza ritrasmissione*, ideale per i dati-sensore. Il canale di controllo/stato resta su uno stream affidabile (stream 0). Poiché il QUIC di INET non implementava i DATAGRAM, sono stati aggiunti al modulo di trasporto.

2 Terminologia

Le parole “datagram”, “frame” e “packet” sono sovraccariche: significano cose diverse a livelli diversi della pila. La Tabella 1 e le successive chiariscono l'uso nel progetto.

Tabella 1: La pila protocollare: una unità per livello.

Livello	Unità	Nella nostra implementazione
App (TODCarApp)	messaggio / <i>sensor frame</i>	SensorDataResponse per sensore; frameId = batch di un tick
QUIC (transport)	QUIC frame	l'unità con i dati: STREAM, DATAGRAM, ACK, MAX_DATA...
	QUIC packet	bundle di frame, dimensionato $\leq$ PMTU, con packet number
UDP	UDP datagram	uno per ogni QUIC packet
IP	IP packet (= IP datagram)	con bit DF $\Rightarrow$ non frammentato
Link (PPP/Eth sim)	link-layer frame	PPP frame

Tabella 2: La parola “datagram” a tre livelli diversi.

Significa	Livello	Nel progetto
QUIC DATAGRAM frame (RFC 9221)	QUIC	✓ QUESTO — la feature implementata
UDP datagram	UDP	il contenitore di un QUIC packet
IP datagram	IP	sinonimo di IP packet

Tabella 3: La parola “frame”, usata in due sensi diversi.

Significa	Livello	Nel progetto
QUIC frame	QUIC	l'unità dentro un QUIC packet (STREAM, DATAGRAM, ACK...)
sensor frame (frameId)	App	batch di dati-sensore di un tick; base del loss_ratio
link-layer frame	Data-link	PPP/Ethernet nel simulatore

Spezzare i dati: tre meccanismi diversi.

- **Segmentazione** (stream) — la fa *QUIC*, automaticamente: un messaggio stream → più STREAM frame su più QUIC packet, riassemblati al ricevitore.
- **Frammentazione IP** — la farebbe *IP* su un pacchetto oversized: QUIC la **evita** (bit DF + dimensionamento a PMTU). Non accade.
- **Chunking** (datagram) — lo fa l'*applicazione*, manualmente: QUIC non frammenta un datagram, quindi è l'app a spezzare il messaggio in pezzi $\leq \text{max_datagram_frame_size}$.

Glossario di progetto.

loss_ratio

frazione di dati-sensore persi per un *sensor frame*: $1 - \text{arrivati/attesi}$. Calcolato da TODAgentApp, inviato a CARLA, usato per la soglia → arresto d'emergenza.

stream 0

lo *stream QUIC* affidabile e ordinato che porta status/controllo.

HoL blocking

head-of-line blocking: in uno stream, un pacchetto perso blocca la consegna dei successivi finché non è ritrasmesso. I datagram lo evitano.

max_datagram_frame_size

transport parameter RFC 9221: dimensione massima di un DATAGRAM frame che il peer accetta (0 = non supportati).

PMTU / DPLPMTUD

Path MTU e il meccanismo con cui QUIC la scopre (RFC 8899) per non far frammentare IP.

3 Architettura: il percorso end-to-end

Il datagram attraversa i seguenti passaggi, dall'invio applicativo fino alla consegna al ricevitore:

```

App:  QuicSocket::sendDatagram(pkt)                                [contract/quic/
      QuicSocket]
      | setKind(QUIC_C_SEND_DATAGRAM)
      v
ConnectionState::processFrame? no -> processAppCommand
      EstablishedConnectionState::processSendDatagramAppCommand
      v
Connection::newDatagram(data)                                     [quic/Connection]
      | verifica max_datagram_frame_size del peer
      | crea DatagramFrameHeader(length) + QuicDatagramFrame -> datagramQueue
      v
PacketBuilder::addFramesFromDatagramQueue (in buildPacket)      [quic/PacketBuilder
]
      | size-aware; scarta se non entra nel pacchetto (no frammentazione)
      v ...UDP / IP / rete 5G lossy... (frame perso => onFrameLost() no-op =>
      NIENTE retx)
RX:  ConnectionState::processFrame -> case FRAME_HEADER_TYPE_DATAGRAM
      v
      EstablishedConnectionState::processDatagramFrame
      | popAtFront(B(length)) <- legge esattamente il payload
      v
Connection::processReceivedDatagram -> AppSocket::sendDatagram [quic/AppSocket
]
      | QUIC_I_DATAGRAM
      v
App:  QuicSocket::ICallback::socketDatagramArrived()

```

4 Modifiche a INET: classi e metodi

Tutte le modifiche sono nel modulo `src/inet/transportlayer/quic` (e nel `contract transportlayer/contract/`). Sono versionate come patch in `tod-omnet/inet-patches/`.

4.1 Nuovo tipo di frame (`packet/FrameHeader.msg`)

- Nuovo valore di enum `FRAME_HEADER_TYPE_DATAGRAM = 0x31` (la variante RFC 9221 “con Length”).
- Nuova classe chunk `DatagramFrameHeader` (estende `FrameHeader`): un solo campo `VariableLengthInteger length` (lunghezza in byte del payload). A differenza dello `StreamFrameHeader` non ha `streamId` né `offset`.
 - `calcChunkLength()` — calcola la lunghezza dell’header: $1 + \text{varint}(\text{length})$ byte.
 - `str()` — rappresentazione testuale `DATAGRAM[length=...]` per il logging.
- Alla classe `TransportParametersExtension` è stato aggiunto il campo `VariableLengthInteger maxDatagramFrameSize` (con aggiornamento di `calcChunkLength()`), per la negoziazione RFC 9221 durante l’handshake.

4.2 `QuicDatagramFrame` (`packet/QuicDatagramFrame.{h,cc}`) — nuova

Rappresenta un DATAGRAM frame. A differenza di `QuicStreamFrame` (che recupera i dati “lazy” da uno `Stream`) tiene il payload *direttamente* come data del frame. Metodi:

- `QuicDatagramFrame()` — costruttore di default.
- `QuicDatagramFrame(Ptr<const DatagramFrameHeader> header, Ptr<const Chunk> data)` — costruisce header+payload; usa chiamate esplicite a `setHeader/setData` (non delega al costruttore base) per far valorizzare correttamente `datagramHeader` tramite dispatch virtuale.
- `setHeader(Ptr<const FrameHeader>)` override — invoca la base e memorizza l’header come `DatagramFrameHeader` (`dynamicPtrCast`).
- `hasData()` override $\rightarrow \text{true}$ — garantisce che il payload venga serializzato nel packet.
- `equals(const QuicFrame*)` override — confronto per *identità*: un datagram è unico e non ritrasmesso, non ha “identità di ritrasmissione”.
- `getDatagramHeader()` — accessor all’header tipizzato.
- **Nessun override di `onFrameLost()`** — è questo che disabilita la ritrasmissione (vedi §5). Eredita inoltre `isAckEliciting()=true` e `countsAsInFlight()=true` dal comportamento di default di `QuicFrame::setHeader`, corretto per RFC 9221.

4.3 `Connection` (`Connection.{h,cc}`)

- Nuovo membro `std::vector<QuicFrame*> datagramQueue` — coda dei DATAGRAM frame in attesa di invio.
- `getDatagramQueue()` — accessor alla coda (usato dal `PacketBuilder`).
- Nel costruttore: `packetBuilder->setDatagramQueue(&datagramQueue)` collega la coda al builder.
- `newDatagram(Ptr<const Chunk> data)` — accoda un datagram per l’invio:
 1. Se il peer non supporta i datagram (`maxDatagramFrameSize == 0`) $\rightarrow$ scarta (RFC 9221).
 2. Crea `DatagramFrameHeader` con `length =` byte del payload e `calcChunkLength()`.
 3. Crea il `QuicDatagramFrame`; se la sua dimensione (header+payload) eccede il `max_datagram_frame_size` del peer $\rightarrow$ scarta.
 4. Se la coda ha già `maxDatagramQueueLength` elementi $\rightarrow$ scarta il **più vecchio** (in testa) per fare posto, ed emette `datagramDropped`. Vedi §4.5.
 5. Accoda in `datagramQueue` e chiama `sendPackets()`; poi emette `datagramQueueLength` (il backlog rimasto dopo il tentativo di invio).

- `processReceivedDatagram(Ptr<const Chunk> data)` — consegna immediata all'app tramite `appSocket->sendDatagram(data)` (nessun reassembly, nessun ordinamento).

4.4 PacketBuilder (PacketBuilder.{h,cc})

- Nuovo membro `datagramQueue` e setter `setDatagramQueue(...)`.
- `addFramesFromDatagramQueue(QuicPacket* packet, int maxPacketSize)` — drena la coda dei datagram nel pacchetto in costruzione, in modo *size-aware* usando `frame->getSize()` (header+payload, a differenza dei control frame che sono solo header). Politica RFC 9221:
 - se un datagram non entra nemmeno in un pacchetto vuoto → viene **scartato** (non si frammenta);
 - se non entra nel pacchetto *corrente* (già parzialmente pieno) → resta in coda per il pacchetto successivo;
 - altrimenti viene aggiunto al pacchetto.
- `buildPacket(...)` — chiama `addFramesFromDatagramQueue` **per ultimo**, dopo control frame e frame STREAM, così i datagram riempiono lo spazio che avanza nei normali pacchetti dati. Vedi §4.5.

4.5 Politica di invio: priorità e coda

I DATAGRAM sono traffico bulk best-effort con una scadenza; gli STREAM portano il canale affidabile. Due scelte, indipendenti dalla RFC ma necessarie perché una sorgente datagram satura non distrugga la connessione:

Gli STREAM hanno priorità stretta sui DATAGRAM. In `buildPacket` i DATAGRAM frame si impacchettano per ultimi. Nell'ordine opposto un frammento datagram da ~1 kB riempie da solo il pacchetto e non lascia mai spazio a un frame STREAM: lo stream affidabile viene affamato a tempo indeterminato. In un sistema di teleguida significa perdere il canale di controllo (status e istruzioni su stream 0) proprio quando la rete è congestionata, cioè quando serve. Il rovescio è che ora è uno stream saturo a poter affamare i datagram: è la politica voluta (*control plane* sopra *sensor plane*).

La coda datagram è limitata e scarta i più vecchi. Parametro NED `maxDatagramQueueLength` (default 64; 0 = illimitata). Un DATAGRAM non è ritrasmesso e trasporta dati con una scadenza: con una coda illimitata, un'app che offre più di quanto il path riesca a portare accumula un backlog che non drena mai e finisce per spedire *solo* dati obsoleti, che il ricevitore scarta come fuori deadline. Scartando il più vecchio, sul link viaggia sempre il dato più fresco.

Misura su `examples/quic/datagram_test`, [Config Overload] (sorgente continua a 17× la capacità del link, nessuna perdita di canale, buffer del router 20 pacchetti): a parità di datagram consegnati (291 in 10s, cioè la capacità del link), con coda illimitata l'ultimo dato consegnato è vecchio di **9,3 s**; con la coda limitata a 8, di **0,56 s**.

Statistiche per connessione: `datagramQueueLength` (backlog dopo ogni tentativo di invio) e `datagramDropped` (datagram scartati per coda piena).

4.6 ConnectionState / EstablishedConnectionState (connectionstate/)

La macchina a stati fa il dispatch dei comandi applicativi e dei frame ricevuti.

- In `ConnectionState::processAppCommand` (switch): nuovo case `QUIC_C_SEND_DATAGRAM` → `processSendDatagramAppCommand(msg)`.
- In `ConnectionState::processFrame` (switch sul tipo di frame ricevuto): nuovo case `FRAME_HEADER_TYPE_DATA` → `processDatagramFrame(...)`.
- `ConnectionState::processSendDatagramAppCommand(cMessage*)` (base, virtual) — lancia “unexpected in the current state” (i datagram si inviano solo da stato Established).
- `ConnectionState::processDatagramFrame(const Ptr<const DatagramFrameHeader>&, Packet*)` (base, virtual) — idem.

- `EstablishedConnectionState::processSendDatagramAppCommand(cMessage*)` override — estrae il `Packet`, chiama `context->newDatagram(pkt->peekData())`.
- `EstablishedConnectionState::processDatagramFrame(header, pkt)` override — legge *esattamente* `length` byte con `pkt->popAtFront(B(header->getLength()))` (un DATAGRAM frame può essere seguito da altri frame nello stesso pacchetto) e chiama `context->processReceivedDatagram(da`

4.7 AppSocket (AppSocket.{h,cc})

- `sendDatagram(Ptr<const Chunk> data)` — costruisce un `Packet` con `kind QUIC_I_DATAGRAM`, vi inserisce il payload e lo invia verso l'app (indicazione dedicata, parallela a `sendData/QUIC_I_DATA` usata per gli stream).

4.8 QuicSocket + QuicCommand.msg (contract) — API applicativa

- `QuicCommand.msg`: nuovo command code `QUIC_C_SEND_DATAGRAM = 9` e nuova indicazione `QUIC_I_DATAGRAM = 12`.
- `QuicSocket::sendDatagram(Packet* msg)` — imposta `setKind(QUIC_C_SEND_DATAGRAM)` e invia al modulo QUIC (nessun tag di stream: il comando dedicato instrada verso `Connection::newDatagram`).
- `QuicSocket::ICallback::socketDatagramArrived(QuicSocket*, Packet*)` — nuova callback, dichiarata *non-pure* con default vuoto (`delete packet;`): le app QUIC esistenti non vanno modificate.
- `QuicSocket::processMessage` — nuovo case `QUIC_I_DATAGRAM` che invoca `cb->socketDatagramArrived(this, packet)` (rispecchia `QUIC_I_DATA`).

4.9 TransportParameters e Quic.ned — negoziazione

- `TransportParameters`: nuovo membro `uint64_t maxDatagramFrameSize`.
- `readParameters(Quic*)` — legge il valore locale dal parametro NED `par("maxDatagramFrameSize")`.
- `readExtension(...)` — legge il valore annunciato dal peer da `TransportParametersExtension::getMaxData`
- `Quic.ned`: nuovo parametro `int maxDatagramFrameSize @unit(B) = default(1200B)` (0 disabilita i datagram). Il valore locale viene serializzato nell'`extension` e inviato durante l'`handshake` da `PacketBuilder`; il valore del peer viene salvato in `remoteTransportParameters` e usato da `Connection::newDatagram` per l'enforcement.

5 Il meccanismo di non-ritrasmissione

La proprietà più importante dei datagram è che **un datagram perso non viene ritrasmesso**. In INET questo si ottiene *senza toccare il gestore di affidabilità*, grazie al polimorfismo di `QuicFrame`.

Quando un pacchetto viene dichiarato perso, per ogni frame contenuto viene chiamato `frame->onFrameLost()` (in `QuicPacket.cc`). Le implementazioni concrete decidono cosa fare:

- `QuicStreamFrame::onFrameLost()` ri-accoda i dati per la ritrasmissione;
- la base `QuicFrame::onFrameLost()` è un **no-op** (corpo vuoto).

Poiché `QuicDatagramFrame` **non** sovrascrive `onFrameLost()`, eredita il no-op: il payload perso semplicemente *sparisce*. In più, il percorso di ritrasmissione nei probe packet (`PacketBuilder::buildAckElicitingPa` filtra esplicitamente solo i frame di tipo `FRAME_HEADER_TYPE_STREAM`, quindi i datagram sono esclusi anche lì. Il datagram resta comunque *ack-eliciting* (contribuisce a RTT e congestion control), come prescrive la RFC 9221.

6 Applicazioni di test e rete

Per la verifica funzionale sono state scritte due app minimali (in `applications/quicapp/`) e una rete con perdita (in `examples/quic/datagram_test/`).

6.1 QuicDatagramSender

Invia N datagrammi numerati a intervalli fissi. Metodi principali: `handleStartOperation` (bind + connect + crea il timer), `socketEstablished` (avvia l'invio dopo l'handshake), `sendOneDatagram` (crea un `ApplicationPacket` con `sequenceNumber` e chiama `socket.sendDatagram`), `finish` (registra lo scalare `datagramsSent`). Parametri: `numDatagrams`, `datagramBytes`, `sendInterval`, `connectAddress/Port`.

6.2 QuicDatagramReceiver

Ascolta, accetta la connessione e conta i datagrammi ricevuti. Metodi principali: `handleStartOperation` (bind + listen), `socketConnectionAvailable` (accept), `socketDatagramArrived` (legge il `sequenceNumber`, logga eventuali *gap* = perdite, conta), `finish` (registra `datagramsReceived`).

6.3 Rete DatagramTest.ned

Topologia `sender - router1 =lossy= router2 - receiver`. Il link centrale è un `PeriodicLossChannel` con parametro `per` (packet error rate) applicato in direzione `sender→receiver`, così gli ACK di ritorno non vengono persi e la perdita colpisce solo i datagram.

7 Risultati della verifica

Esecuzione con **20% di perdita** sul link centrale, 20 datagrammi da 800 B:

Datagrammi inviati	20
Datagrammi ricevuti	16
Persi (mai ritrasmessi)	seq 2, 7, 13, 19
Dopo ogni "buco"	il seq successivo arriva <i>subito</i>

Sequenza ricevuta: 0,1,_,3,4,5,6,_,8,9,10,11,12,_,14,15,16,17,18. Questo dimostra sperimentalmente tutte le proprietà attese:

- **Consegna** funzionante (16/20 arrivati: il percorso `app→frame→RX→app` gira);
- **Inaffidabilità** (4 pacchetti persi per bit-error sul link);
- **Nessuna ritrasmissione** (i seq persi non riappaiono mai — con gli stream sarebbero stati ritrasmessi);
- **Nessun head-of-line blocking** (dopo il buco su 2, il seq 3 arriva immediatamente);
- **Negoziazione** corretta (con `max_datagram_frame_size=0` sarebbero stati scartati tutti; invece 16 sono passati $\Rightarrow$ il peer ha annunciato supporto).

8 Build ed esecuzione

Il workspace è gestito da `opp_env` (ambiente Nix riproducibile): il build va fatto entro la sua shell.

```
# build incrementale di INET (rigenera i makefile per i nuovi .cc, ~30s con
  ccache)
opp_env run -w . --no-build -c 'make -C inet-4.6.0 MODE=release -j 4'

# esecuzione del test funzionale
opp_env run -w . --no-build -c 'cd inet-4.6.0/examples/quic/datagram_test && \
  opp_run_release -u Cmdenv \
    -n <INET>/src:<INET>/examples \
    -l <INET>/out/clang-release/src/libINET.so omnetpp.ini -r 0'
```

Note: buildare dalla *radice* di INET (non da `src/`) affinché `make` `makefiles` rigeneri la lista OBJJS e includa i nuovi file. Nell'`omnetpp.ini` il livello di log va per-oggetto (`**cmdenv-log-level=info`).

9 Versioning e prossimi passi

Poiché INET è una dipendenza pinnata (non un repository git), tutte le modifiche sono conservate come **patch versionata** nel repository `tod-omnet`: `tod-omnet/inet-patches/quic-datagram.patch` (trasporto) e `tod-omnet/inet-patches/test/` (app di test e rete), con uno snapshot pristino in `orig/` per rigenerare il diff.

Fase 2 (integrazione applicativa in `tod-omnet`).

- `TODCarApp` invia i dati-sensore via `sendDatagram` (mantenendo uno `streamId` “nominale” nel payload per identificare il sensore), spezzandoli in chunk $\leq \text{max_datagram_frame_size}$;
- un *manifest* sullo stream 0 affidabile dichiara, per ogni sensore, numero di datagram e byte;
- `TODAgentApp` calcola un *loss_ratio pesato per-sensore*;
- un interruttore di configurazione permette il confronto A/B tra “datagram inaffidabile” e “stream affidabile”.